INDIAN INSTITUTE OF TECHNOLOGY, ROORKEE

Department of Computer Science and Engineering

CSC 206 - SOFTWARE ENGINEERING

Kernel-Level System Performance Monitor

using eBPF

Submitted By: Group 11

Team Members

Name	Roll No.	Email ID
Vaibhav Kumar	24114103	vaibhav_k@cs.iitr.ac.in
Samarth Maheshwari	24114084	samarth_m@cs.iitr.ac.in
Vishal Kumar Shaw	24114105	vishal_ks@cs.iitr.ac.in

Individual Contributions

Name	Contribution
Vaibhav Kumar	*Implemented kernel-space eBPF programs, including tracepoint handling, BPF maps, and ring buffer communication. Also developed the statistics engine featuring latency computation, P95 histograms, EMA baselines, and anomaly detection logic.* .
Samarth Maheshwari	*Designed the overall system architecture, coordinated integration between kernel-space and user-space modules, and contributed to core development, testing, debugging, and project documentation.*
Vishal Kumar Shaw	*Developed the dashboard interface with multi-panel TUI visualization, handling layout, theming, and real-time rendering. Implemented data export modules for JSON/CSV, anomaly logging, and integration for external analysis tools.*

1. Introduction

1.1 Background Concepts

In this project **eBPF** (Extended Berkeley Packet Filter) is used which is a Linux kernel technology that allows small, safe programs to run directly inside the kernel without modifying it. Before running, every eBPF program is verified by the kernel for safety ensuring no crashes, bounded loops, and valid memory access then JIT-compiled to native machine code for near-native performance. Three core building blocks make this possible.

1. Tracepoints
Predefined hooks inside the Linux kernel where eBPF programs attach to observe events (like syscall entry/exit, scheduling, process execution, etc.) in real time without pausing execution. This provides **zero-interruption observation** i.e. the system keeps running at full speed (unlike debugger which interrupts the system).

2. BPF Maps
Shared key-value storage accessible from both kernel and user space. Stores timestamps, per-thread state, and intermediate metrics during event processing. This enables **stateful tracking across kernel events** meaning the system retains information from earlier events (such as syscall entry timestamps) and uses it when processing later related events (such as syscall exit), without crossing the kernel boundary on every single event.

3. Ring Buffer
An efficient FIFO data structure which moves structured events from kernel space to user space without using locks and has a fixed size. It has almost **near-zero latency and ordered delivery** with minimal memory overhead. Thus, it is safe for high-frequency production workloads.

1.2 Motivation

Modern Linux systems run thousands of processes simultaneously, each making hundreds of system calls per second. Existing monitoring tools each cover only part of the picture, and none provides the complete view needed for production-grade diagnostics.

Tool	What It Shows	Overhead	Key Limitation
top / htop	CPU %, memory, process list	< 1%	No syscall latency or kernel detail
strace	Every syscall with arguments	100–1000 times	Unusable in production
perf	Hardware counters, aggregate stats	2–5%	No live dashboard; needs expertise
sysdig	System call events with context	3–10%	No adaptive anomaly detection
Our tool	**Per-process, per-syscall latency + anomaly detection**	**< 3%**	Limited scalability (512 tracked entries)

Table 1: Existing tools and their limitations

1.3 Problem Statement

System administrators, developers, and security engineers often need a way to understand what is really happening inside a running system. So, we need a system that would do the following task simultaneously:

- Observe the processes behaviour at very fine level especially, CPU scheduling, I/O activity with precise time.
- Provide latency metrics such as averaged P95, maximum values in near real time without needing large amounts of raw data.
- Detects and highlights unusual behaviour of a process when it behaves differently in accordance to an adaptive threshold.
- Keeps the monitoring overhead minimal so that the system performance does not get affected.
- Collects and exports the metrics data in a structured form to User level for analysis or any other task.

Currently, there exists different tools that address some parts of the stated problem but none of them provide all of these functionalities in together. This project aims to create such a system that offers detailed visibility, near-time analysis and low overhead in one place.

Core Problem: How can we design a system which collects per-process events such as syscall, latency, context switches, scheduling in LINUX kernel and at the same time ensuring minimal overhead and anomaly detection in real time.

1.4 Application Domain

This tool is designed for the organisations which needs a monitoring system for identifying performance bottlenecks or diagnose operational issues such as :

- **Site Reliability Engineering (SRE):** When a web server like nginx or Apache faces high traffic and thus slows down, this tool helps to pinpoint the potential issues such as disk I/O delays, scheduling or something else.
- **Database Administration:** for system running Databases like MySQL, the overall performance gets degraded due to several reasons such as increase in read/write latency or query processing.
- **DevOps & CI/CD:** Many subprocesses run in parallel while deploying a software. This tool helps in keeping a track of resource utilisation by different processes.
- **Security Monitoring:** There can be unusual spikes in system call or system performance signalling a security issue. Using this tool there will be a track of system behaviours and avoid such issues.
- **Research & Education:** This tool provides a practical platform for learning the Operating system behaviour in real time. Providing visibility into the costs of system calls or scheduling decisions.

2. Proposed System

2.1 System Architecture

The architecture is made up of two different layers: the kernel layer and the user layer. This ensures that the collection of data is done very easily and that data can even be processed externally to the kernel. The result is shown live on the dashboard of the terminal.

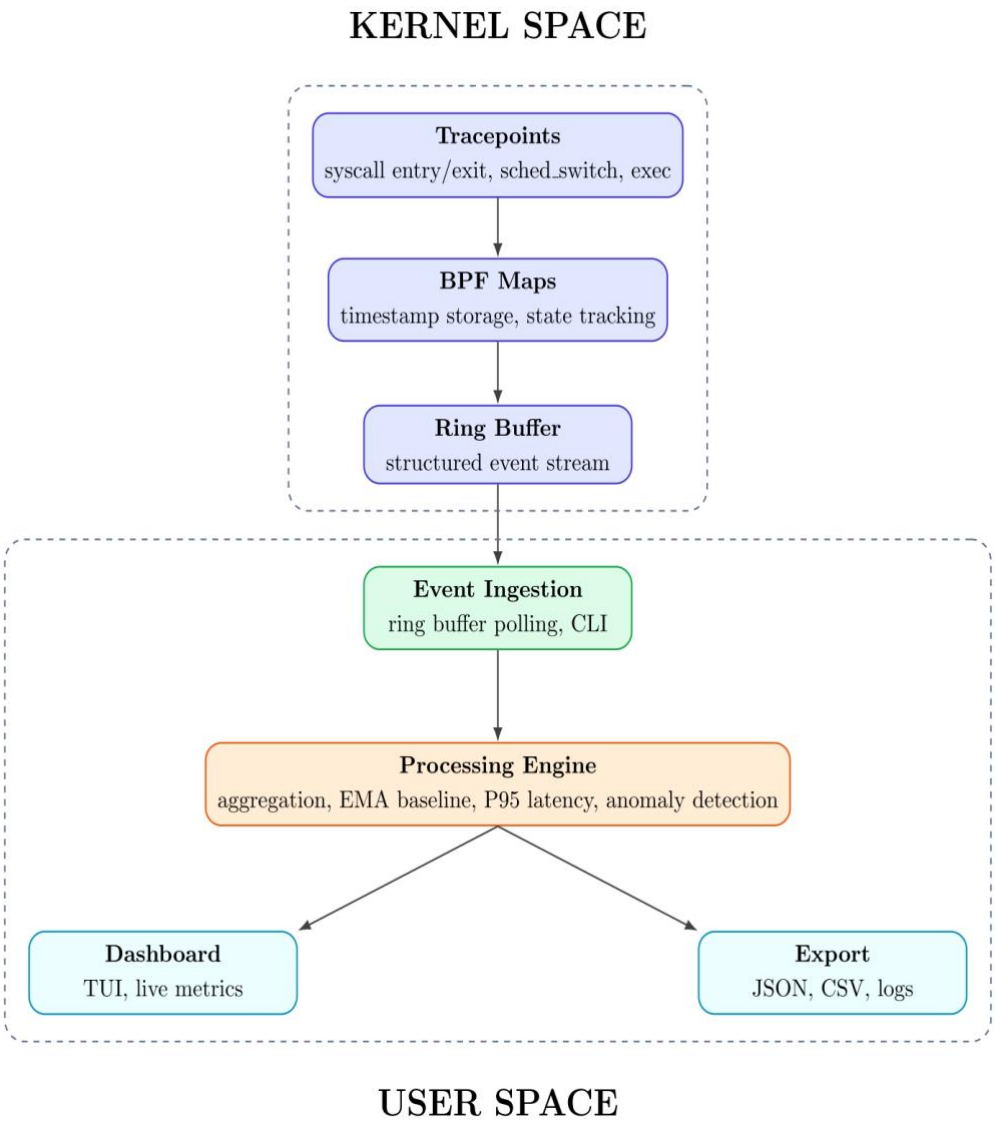

Figure 1: High Level Architecture of the System

2.2 System Components

The architecture is divided into two parts: one to collect data known as the kernel layer and another to analyze data and visualize data which is the part of the user space layer. This clear difference of roles makes the system run efficiently.

Kernel Space Components

The kernel layer in the kernel space handles the collection of system events.

Component	Role & Description
Tracepoints	The process starts from the tracepoint, which detects events like system calls, scheduling, and process execution. Once the event happens, the eBPF program will capture the timing details required.
BPF Maps	In order to connect two events, the framework keeps track of the middle event by using BPF maps, which store the timestamp data. This will enable connecting the events, and then the required metrics such as latency can be determined.
Ring Buffer	With sufficient data acquired, the event is organized and delivered to the ring buffer, which serves as the interface for transferring from kernel space to user space thus ensuring that events are transferred seamlessly and without any hindrance.

Table 2: Kernel space components and their functions

Together, they make up a light-weight pipeline inside the kernel, focusing only on collecting and delivering data.

User Space Components

The user-space layer collects the events from the ring buffer and then processes it..

Component	Role & Description
Event Ingestion	The first stage continuously reads and filters the incoming data stream. This ensures only relevant processes and events proceed to the processing stage.
Processing Engine	Aggregates and analyzes events to compute latency metrics and track how process behavior changes over time. Unusual patterns that may indicate performance issues are identified here.
Dashboard & Data Export	Processed results branch into two outputs: a live TUI dashboard for real-time monitoring, and structured file exports (JSON/CSV) for offline analysis and integration.

Table 3: User space components and their functions

2.4 System Workflow

The program begins with the deployment of the monitor, which involves the loading and attachment of eBPF programs, as well as the setting of filters. During execution, the kernel will keep generating tracepoints and updating BPF maps, which will be added to the ring buffer (through the main loop as shown in Figure 2). In the user space, these tracepoints will be analyzed to calculate the metrics and identify any anomalies.

The workflow is illustrated as shown:

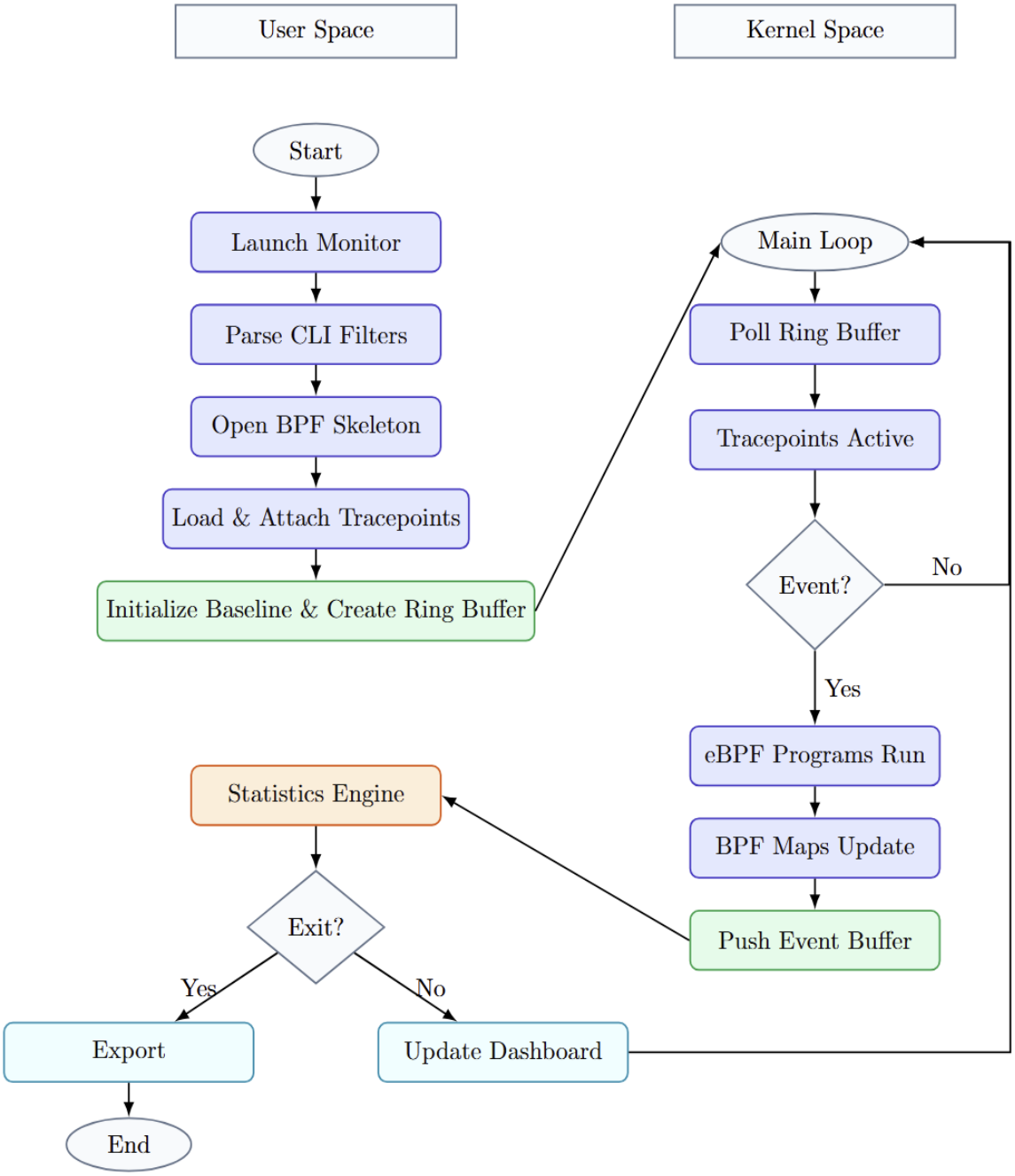

Figure 2: Workflow of the System

2.5 Algorithms

Below are listed the algorithms that form the core analysis engine of our system. These algorithms are designed for efficiency in a real-time environment, with calculations being done in constant time per event; integers are used in place of floats wherever possible, and memory consumption is limited and all computations are completed in constant time per event.

Algorithm 1: System Call Latency Measurement

Objective: Measure the exact duration of each system call without interrupting execution.

Step 1 — Record Entry Timestamp

When a system call begins, the eBPF program fires at the syscall_enter tracepoint. A nanosecond timestamp t_enter is stored in a BPF map, keyed by the current thread ID (tid).

Step 2 — Compute Latency on Exit

When the matching exit event fires at syscall_exit, the stored entry timestamp is retrieved and the latency is computed:

$$Latency = t_{exit} - t_{enter}$$

Design constraints: Timestamps are managed separately for each thread which means that there can be just one system call active at any time for each individual thread. Only matching pairs of events are taken into account,otherwise they are ignored to ensure data accuracy.

Algorithm 2: Scheduling Delay Measurement

Objective: Quantify how long a process waits between being scheduled off the CPU and back on — a direct measure of CPU contention.

Scheduling delay is approximated as the time between two successive CPU executions of a process, using context-switch events:

$$Scheduling\ Delay = t_{schedule\ in} - t_{schedule\ out}$$

Noise Filtering: Delays below 200 ns are filtered out to eliminate noise from brief idle transitions. These filtered samples are still counted toward rate tracking, preserving throughput accuracy while removing measurement skew.

Algorithm 3: EMA-Based Baseline & Anomaly Detection

Objective: Automatically detect when a process deviates from its normal behavior — without requiring manual threshold configuration.

The system maintains a dynamic baseline per (process, event) pair using an **Exponential Moving Average (EMA)**, which gives more weights to recent observations while also considers previous observations which helps in calculating fair criteria for baselines.

Step 1 — Update the Baseline

On each new observation x_t, the EMA baseline B_t is updated with smoothing factor α = 0.2:

$$B_t = \alpha \cdot x_t + (1 - \alpha) \cdot B_{t-1}$$

Step 2 — Compute Relative Deviation

The deviation D measures how far the current observation is from the established baseline, expressed as a fraction:

$$D = \frac{(x_t - B_t)}{B_t}$$

Step 3 — Flag Anomaly

If the deviation exceeds the threshold θ = 0.50 (i.e., 50%), the event is flagged as anomalous:

$$Anomaly = 1;\ if\ D > \theta, else\ 0$$

Implementation note: All integer operations have been used to maintain compatibility with the kernel. The EMA maintains the baseline for each process separately, thus a slow database operation will not affect the baseline of an I/O process which is fast.

Algorithm 4 — Approximate P95 via Histogram Buckets

Objective: In real-time measurement scenarios, maintaining constant memory storage for each individual sample needed to calculate the 95th percentile latency is difficult.

This solution makes use of a fixed-size, **multi-bucket histogram** for each (process, event) combination, where each statistic entry consumes 64 bytes of memory regardless of the number of events.

Bucket	Latency Range	Typical System Calls
0	< 10 µs	Cached reads, simple memory-mapped ops
1	10–50 µs	Standard file reads, socket polls
2	50–100 µs	Uncached reads, moderate I/O
3	100–500 µs	Disk seeks, heavy write operations
4	500–1000 µs	Slow disk access, network calls
5	1–5 ms	High-latency I/O, lock contention

6	> 5 ms	Severe bottlenecks, blocking calls

P95 Estimation Procedure

Cumulative counts are added across buckets moving from lowest to highest. The P95 value is reported as the upper edge of the first bucket whose cumulative count reaches or exceeds 0.95 × N_valid total samples. The result is capped by the maximum observed latency to avoid overestimation in sparse cases.

Unit note: All latency values are measured in nanoseconds inside the kernel and converted to microseconds before bucket classification for human-readable reporting.

2.6 Novelty, Innovations and Decisions

The system utilizes a collection of intentional design decisions, which help distinguish it from simple eBPF applications.

1

Filtering Inside The Kernel

Problem solved: Unnecessary data was being sent to user space

It solves a common problem by moving event filtering into the kernel space in order to avoid transferring every raw event across kernel/user boundary. In this way, process identifiers (PIDs) and process names can be provided as a compile time constant before loading the eBPF program, thus preventing the transfer of mismatching events outside the kernel.

2

Scheduling Noise Isolation

Problem solved: Very small delays were adding useless noise to stats

It helps to reduce skew caused by extremely fast (faster than 200 nanoseconds) context switching. Such samples do not contribute to measurements and are filtered out; however, they still get accounted in rates calculations. In this manner, throughput gets correctly measured without noise.

3

Cell level terminal rendering

Problem solved: Screen was flickering due to constant full refresh

It solves issues related to flicker and additional computations resulting from redrawing the entire terminal. A new snapshot of a virtual buffer is diffed against the previous one and only differences are updated using 24 bit color sequences, producing smooth and responsive TUI at an interval of 2 seconds.

4

Separate Summation Of Outliers

Problem solved: Big spikes were hiding normal behavior

It solves problems associated with a single outlier hiding other, real trends in data by being included in the calculation of aggregate average. Thus, a separate representation of outliers in summary views allows detecting both degradation and spikes.

5

Accurate Measurement of Overhead

Problem solved: Initial setup overhead was giving wrong high readings

It prevents the problem of overestimated results caused by significant one-time overhead of just-in-time (JIT) compilation and loading of an eBPF program. Samples are collected only when a process reaches steady state, i.e., the initial overhead does not affect results.

3. Implementation

3.1 Tools and Technology

Technology / Tool	Purpose
Linux kernel	eBPF program runtime environment
Clang / LLVM	Conversion of eBPF C source code to BPF bytecode
libbpf	BPF skeleton creation, CO-RE relocation, ring buffer interface
bpftool	Creation of skeleton headers management of eBPF programs, maps, and links
libelf / zlib	Transitive dependencies of libbpf library
GNU Make	Automatically organizing and building the system components
vmlinux.h	Kernel data type definitions of all types needed for CO-RE
GCC	Compilation of C11 user space source code

Table 4: Technology stack and purpose

3.2 Module Dependency

We have used four independent modules to build our system. The diagram illustrates the dependency and data-flow relationships amongst them.

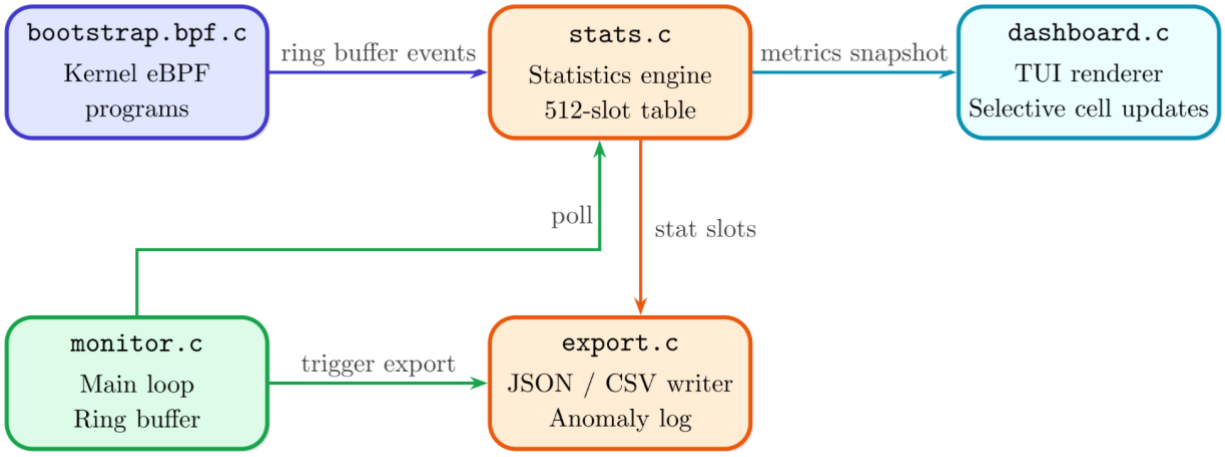

Figure 3: Module Dependency

3.3 Module Implementation Details

Build System Portability

The Makefile automatically detects the architecture of host CPU and maps that to bpf target. It enables compilation on x86 _64, ARM and more platforms without manual reconfiguration.

BPF Program Compilation and Loading

The project uses a three-phase build process pipeline. In the first phase, Clang/LLVM translates the kernel eBPF source code to BPF bytecode object files. In the second phase, bpftool creates a skeleton header that embeds the bytecode in a byte array format along with the APIs of maps and program handles. In the third phase, GCC builds the user-space application and links it to libbpf.

During execution, the user space application loads the embedded BPF object based on PID/name filters. Afterward the kernel verifies its safety and bound execution before attaching the programs to the tracepoints.

Unified Event Structure

The ring buffer holds only one event structure of a fixed size that carries all the events that require monitoring. The event structure comprises process ID, timestamp information, event type, process name, and an arbitrary field of text used interchangeably to refer to system calls, scheduler types, termination of processes, or executable paths.

Statistics Engine

The monitor maintains statistics in a simple array consisting of 512 records, each corresponding to a (program name, event type). Linear search is used since there are few hundreds of entries involved, and hence it will be quicker to perform a linear search than using a hash table. All entries can be cleared by setting memory to zero once.

Adaptive Dashboard Rendering

The display adjusts dynamically according to the width of the terminal being used, with additional diagnostic columns becoming available as the space available grows larger:

- **< 70 cols:** Minimal display like PROCESS, EVENT, RATE, AVG, MAX
- **≥ 70 cols:** Includes mini rate bar (visual representation, eight characters)
- **≥ 80 cols:** Includes PID (process ID) column
- **≥ 95 cols:** Includes P95 (percentile) column
- **≥ 115 cols:** Diagnostic display includes CTXSW and EXECS columns

The layout engine automatically resizes the panels according to the dimensions of the terminal when there is a refresh.

3.4 Dashboard Layout

The terminal dashboard contains multiple panels encircling the status header in the center. The status header shows the run-time, CPU usage, rate of events, buffer usage, and number of dropped events.

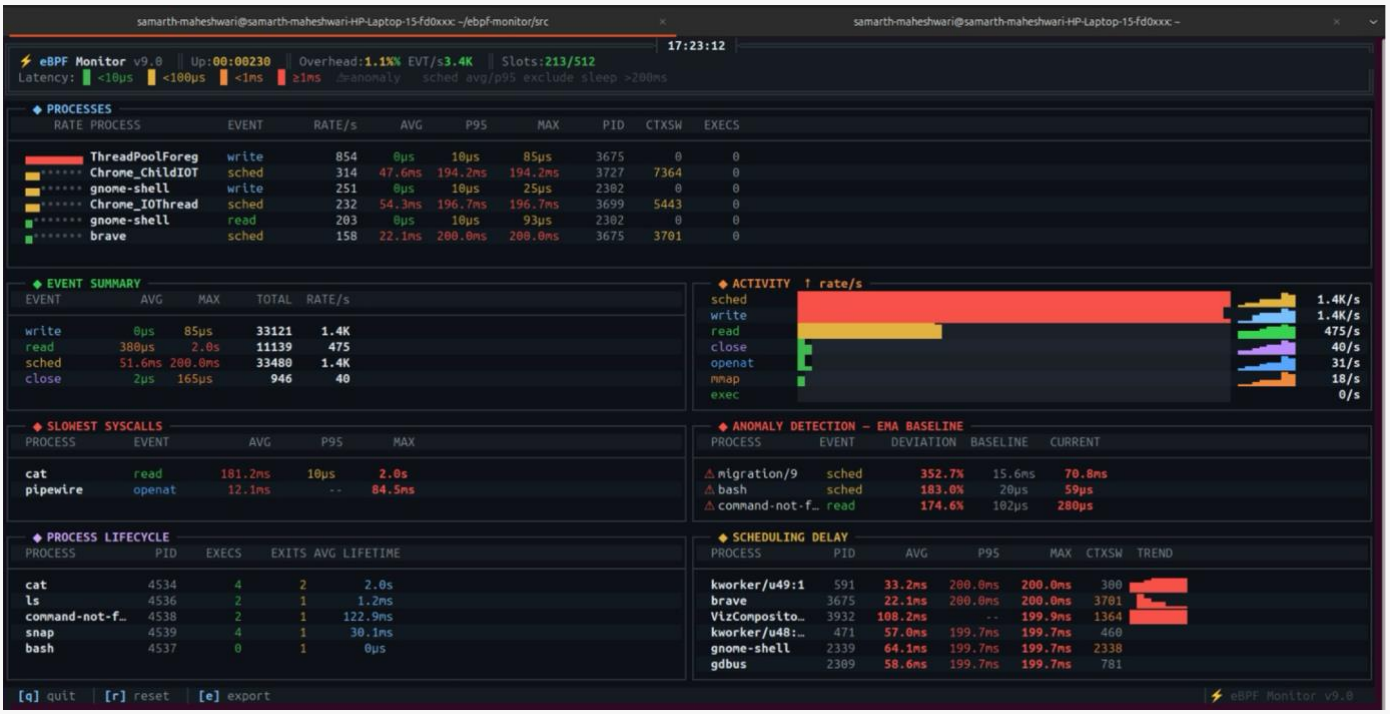

Figure 4: Dashboard Layout

Contents of Dashboard:

#	Panel	What It Shows

(i)	**Processes Panel**	Process wise statistics: event rate, avg/P95/max latency, context switches, exec count.
#	**Panel**	**What It Shows**
(ii)	**Event Summary**	High-level system wide aggregations that provide an overview on average system wide latencies.
(iii)	**Activity Panel**	Live graph indicating the syscall type categories with the largestpresence in the current activity of the system.
(iv)	**Slowest Syscalls**	Indicates the process and syscall with the largest latencies observed this indicates performance problems directly.
(v)	**Anomaly Detection**	Lists processes whose behavior deviates from their EMA baseline, with deviation magnitude.
(vi)	**Scheduling Delay**	Displays per process scheduling latencies, distinguishing between I/O waits and CPU competition.
(vii)	**Process Lifecycle**	Monitor process lifecycles (creation, deletion), average lifetime.
(viii)	**Interactive Controls**	Key shortcuts: search by PID/Name, Export button, Reset Statistics, Quit.

Table 5: Dashboard panels and their content

3.5 Export Format

JSON export: Full record of all the statistics recorded by the tool, on exit/e keypress. The JSON entry is of format: process, pid, event, count, rate, avg/p95/max latency, context switch count, number of executions, baseline latency, percentage of deviation from baseline latency, anomaly flag.

CSV export: The same data as a flat CSV with a header row, directly importable into pandas, Excel, or R for batch analysis.

Anomaly log: A CSV file written out once every two seconds as a new frame is drawn. Includes session start/end flags and one row per anomaly detected, consisting of the following fields: time stamp, process, event, percentage of deviation from baseline latency, baseline latency, current average latency.

4. Software Engineering Principles

4.1 Development Process: Evolutionary SDLC

Feasibility Study: The feasibility of constructing a kernel level monitoring system using eBPF was considered during the feasibility study. Technical feasibility, risk, resource analysis, and low overhead real-time monitoring were the factors that were considered at that point.

- Finished on 08.03.2026.

Software Requirement and Specification: Here we discussed the requirements of the software such as the functional and non-functional requirements, customer expectations, and behaviors of the software including kernel events, latency, and visualization through CLI.

- Finished on 23.03.2026.

Design: This involved designing the architecture of the software, which included the data flow within the system where eBPF programs, BPF maps, ring buffer, and other user-space elements are discussed.

- Finished on 07.04.2026.

Evolutions: The software was made in 4 evolutions as shown below:

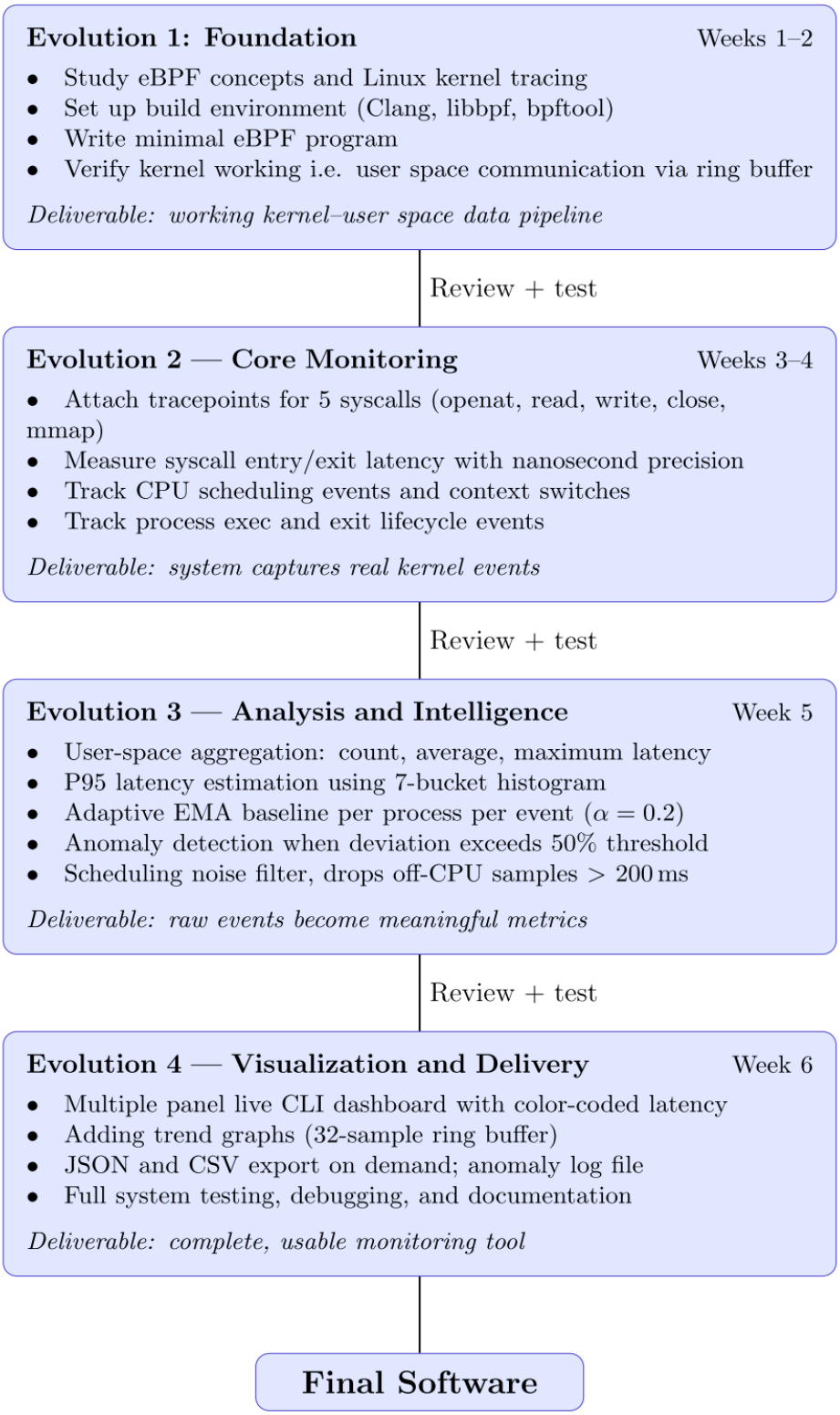

Figure 5: Evolutions of the System into Final Software

4.2 Modular Design

The system is deliberately split into four independent modules each with a single responsibility so that any one module can be modified or replaced without affecting the others.

- bootstrap.bpf.c handles kernel eBPF programs and tracepoint registration. A new syscall here means automatic adaptation of stats, dashboards, and exporter modules.
- stats.c owns the 512-slots flat statistics table. Any changes to the aggregation algorithm need no modification in any exporting module.
- dashboard.c includes the cell level TUI visualizer and panel organization logic. Redesigning visualization would not affect any data collecting or exporting process.
- export.c handles exporting into JSON/CVS and logs anomalies. Other export types can be implemented here without changing anything else.

4.3 Pair Programming

For some of the more challenging aspects of development, such as eBPF programs and ring buffer communication, we split the group into three pairs of two people each where one person wrote the code and other focuses on spotting any errors that may occur. In particular, this proved crucial for kernel space coding due to the lack of feedback when the code failed verification by the kernel.

4.4 Version Control and Collaborative Development

All aspects of the assignment were handled through a single GitHub repository(i.e through a single account) where all the coding was done on a common system and account. The team members made local changes to the code, synchronize the codes, and ensure they do not conflict with each other before adding the code to the repository. All changes were made systematically in the form of commits after internal discussion and basic verification.

5. Comparison with Existing Systems

5.1 Comparison Table

Feature	eBPF Monitor	top / htop	strace	perf stat	sysdig
Monitoring overhead	< 3%	< 1%	100–1000×	2–5%	3–10%
Per-syscall latency	Yes (5 syscalls)	No	Yes (all)	Aggregate only	Yes
Per-process granularity	Yes	Yes	Yes (one proc)	Partial	Yes

Feature	eBPF Monitor	top / htop	strace	perf stat	sysdig
CPU scheduling delay	Yes (noise-filtered)	No	No	Indirect	Yes (raw)
Adaptive anomaly detection	Yes (EMA + 50%)	No	No	No	Rule-based only
P95 latency (per process)	Yes (histogram)	No	Manual calc	Yes (global)	Yes
Live TUI dashboard	Yes (7 panels, 2s)	Yes	Scrolling log	Static report	Basic
Kernel-side filtering	Yes (zero overhead)	Yes (PID)	Yes (PID)	Limited	Yes
Production safe	Yes	Yes	No	Limited	With caution
Kernel modification needed	No	No	No	No	No
CO-RE portability	Yes	Yes	Yes	Yes	Partial
Structured export (JSON/CSV)	Yes (both)	No	Log file	Text report	Yes
Continuous anomaly log	Yes (per-cycle)	No	No	No	Alerting only

Table 6: Comparison of eBPF Kernel Monitor with Traditional Monitoring Tools

5.2 Discussion

strace makes use of the ptrace() functionality, which means that the tracked process is stopped before and after each syscall is made. This technique involves high overhead (on average between 2x and 200x), is limited to one PID and hardly can be used for production purposes. At the same time, an eBPF monitor works with kernel tracepoints on the execution path without interrupting the tracked process.

top/htop collects the information using /proc file system at specific time periods and display average CPU load and memory consumption. In addition, these tools have no means of providing a metric describing syscall latency, separating different types of I/O operations, getting metrics per syscall, and distinguishing between processes which are slow from those which are just busy.

perf stat tool is quite powerful, but it needs to know PIDs in advance and aggregates the results during the whole interval of work. In addition, it does not offer percentile syscall latencies, does not provide live dashboards and does not perform anomaly detection.

sar collects information about system performance without linking metrics to PIDs and therefore it cannot show latency by process and syscall.

Advantages of this approach:

- Generates P95 latency for each process, each system call.

- Uses an adaptive method of detecting anomalies based on moving average baselines.

- Can trace all processes simultaneously at the cost (<3%) overhead.

- Features kernel-level filters preventing unnecessary information from going to user-space, thus minimizing overhead associated with transferring and processing the data.

- Offers structured exports (JSON/CSV), suitable for further analysis and alerts.

6. Conclusion

6.1 Key Outcomes

The research proves the feasibility of creating a concise, one-file-based program using eBPF for monitoring the entire system in real-time mode without causing any performance degradation or system changes within the kernel. The main accomplishments include the following:

- **Precision**: The program calculates the time spent on each system call accurately, even when there are several threads running simultaneously.
- **Minimal impact**: The program uses only about 1% of CPU resources on average and does not exceed 3% during busy periods of work, whereas programs like strace may cause up to 100–200% degradation in system performance. It can be used permanently.
- **Smart detection**: The program establishes normal behavior for each process and triggers an alarm signal only in case of abnormal events, while ignoring false alarms related to the inactivity of processes.
- **Convenient presentation**: The real-time interface includes several screens with color information, graphics, and statistics related to specific processes. It provides details similar to those offered by more advanced and costly applications.
- **Portability**: The application can be used on different Linux versions and does not need recompilation for every new version of the OS.
- **Quality of the source code**: The source code looks good in terms of its readability and clear division into logical parts.

6.2 Practical Usefulness

Fast problem solving: With the decrease in server performance, this tool will be able to instantly find the process causing this issue, define which particular system call causes the excessive time and detect whether this behavior is suspicious without the need to restart anything.

Development support: Developers can use this tool in the process of writing software to figure out how many system calls their code makes and how fast they are executed.

Security monitoring: The ability to automatically discover and log sudden changes in file scanning rate or abnormal activity or performance of processes becomes available after that.

Educational purposes: Since all parts of this utility have plenty of comments, it is useful not only for practical purposes but also to learn more about the way eBPF works or the Linux kernel handles system calls.

6.3 Future Improvements

1. Increase in types of monitored system calls: At the moment, the tool supports system calls for files and I/O. The proposed solution assumes that it is possible to expand the monitoring to include network related system calls (e.g. connections and packages) as well as thread creation besides the already available options.

2. Containerization: Make monitoring more efficient by organizing data based on Docker containers or Kubernetes pods, which will make it possible to monitor applications within the cloud environment.

3. Dynamic change in filter settings: Allow for changing monitoring settings concerning processes on the fly without shutting down the application.

4. Logging capabilities: Make sure that the tool can integrate with logging systems like Prometheus or InfluxDB in order to keep track of trends over time instead of monitoring data at particular moments in time.

5. Improved process filtering options: At the moment, only exact matching on process name is allowed. However, being able to use partial matching and patterns for identifying interesting processes will prove useful (e.g., identifying all Java processes).

6. User Interface: Build a web application to monitor the system remotely without connecting to each machine individually.

7. Bibliography

[1] B. Gregg, BPF Performance Tools: Linux System and Application Observability. Addison-Wesley Professional, 2019.
[2] B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. Addison-Wesley, 2020.
[3] The Linux Foundation, "What is eBPF?" Available: https://ebpf.io/what-is-ebpf/. Accessed April 2026.
[4] The Linux Kernel Organization, "BPF Documentation." Available: https://www.kernel.org/doc/html/latest/bpf/index.html. Accessed April 2026.
[5] A. Nakryiko, "BPF CO-RE Reference Guide," 2021. Available: https://nakryiko.com/posts/bpf-core-reference-guide/. Accessed April 2026.
[6] A. Nakryiko, "BPF Ring Buffer," 2020. Available: https://nakryiko.com/posts/bpf-ringbuf/. Accessed April 2026.
[7] libbpf project, "libbpf: BPF Portability and CO-RE." Available: https://github.com/libbpf/libbpf. Accessed April 2026.
[8] kernel.org, "Tracepoints documentation." Available: https://www.kernel.org/doc/html/latest/trace/tracepoints.html. Accessed April 2026.
[9] Facebook / Meta, "bpftool — tool for inspection of eBPF programs and maps." Available: https://github.com/libbpf/bpftool. Accessed April 2026.
[10] BCC authors, "BPF Compiler Collection (BCC)." Available: https://github.com/iovisor/bcc. Accessed April 2026.

[11] B. Gregg, "Linux Extended BPF (eBPF) Tracing Tools." Available: https://www.brendangregg.com/ebpf.html. Accessed April 2026.
[12] D. Vogt, F. Bellard, and T. Gleixner, "Tracing the Linux Kernel with eBPF," Proc. Linux Plumbers Conf., 2019.
[13] Sysstat project, "sysstat: System performance tools for Linux." Available: http://sebastien.godard.pagesperso-orange.fr/. Accessed April 2026.
[14] C. Canevet and A. Bailleu, "Continuous profiling in production with eBPF," Proc. SREcon EMEA, USENIX, 2022.